

unuser

UniqueUser — ambiente de sincronização hierárquica com cofre cifrado

Especificação técnica · v0.4 · 03/06/2026

1. Objetivo

O **unuser** sincroniza documentos e arquivos de trabalho entre várias máquinas Debian (mesma versão), substituindo o fluxo atual de cópia manual por pendrive. A sincronização é **manual e controlada pelo usuário** através de um aplicativo gráfico inspirado no **Windows XP Explorer**, e todo o conteúdo é protegido por criptografia ponta a ponta: o servidor central é um **cofre cego** que nunca tem acesso ao conteúdo, aos nomes dos arquivos nem à estrutura de pastas.

2. Conceitos

Termo	Definição
Cofre	O conjunto de arquivos sincronizados, protegido por passphrase + keyfile.
root key	Chave-mestra derivada no cliente; nunca sai da máquina nem chega ao servidor.
Manifesto	Índice cifrado de um arquivo: metadados, a FK embrulhada e, por versão, o carimbo de data/hora (até o segundo), o hash e as referências aos blocos.
Blob	Bloco de conteúdo cifrado e endereçado por identificador opaco, armazenado no servidor.
Cofre cego	O servidor: guarda apenas blobs e manifestos cifrados; não consegue ler nada.
Onion Service	Serviço oculto do Tor (v3) pelo qual o servidor é alcançado fora da LAN.
FK (chave de arquivo)	Chave aleatória exclusiva de cada arquivo, usada para cifrar o conteúdo dele.
KEK	Chave-mestra do usuário (derivada de passphrase+keyfile) que "embrulha" as FKs. É a "chave geral".

3. Decisões de arquitetura

Item	Decisão
App cliente	GUI em PySide6/Qt , visual inspirado no Windows XP Explorer (tema Luna) : painel de tarefas lateral, árvore de pastas, status e ações por arquivo.
Servidor	Cofre cego (<i>zero-knowledge</i>): blobs + manifesto cifrados + histórico de versões.
Linguagem	Python ≥ 3.11.
Topologia	Hub-and-spoke: servidor central sempre ligado (fonte da verdade) + clientes.
Rede	Mista. Na LAN: conexão direta por IP. Fora da LAN: Tor .
Conexão	IP direto ou Tor — selecionável na própria interface.
Trânsito	mTLS (TLS mútuo). Fora da LAN, o mTLS roda sobre o Onion Service do Tor. Sem VPN.
Repouso	LUKS já existe no SO de cada máquina (fora do escopo do unuser). No servidor, o conteúdo permanece cifrado pelas chaves do cliente.
Sincronização	Manual, disparada ao abrir o cofre e por botão "atualizar". Sem daemon de watch.

4. Modelo de segurança

4.1 Hierarquia de chaves (criptografia em envelope)

O cofre só abre com **passphrase + keyfile** (os dois são obrigatórios). O keyfile é um arquivo de alta entropia que pode morar no pendrive — assim o pendrive deixa de carregar os arquivos e passa a carregar apenas a chave. Cada arquivo é cifrado com uma **chave aleatória própria (FK)**; a chave-mestra do usuário (**KEK**) apenas *embrulha* essas FKs.

```
passphrase —Argon2id(salt)→ kp
keyfile (alta entropia)      → kf
root_key = HKDF-SHA256( ikm = kp || kf )

root_key —HKDF(label)→ KEK          (embrulha as chaves de arquivo)
                        name_key      (cifra nomes e metadados no manifesto)
```

manifest_key (cifra o manifesto)

Por arquivo:

FK = aleatória de 256 bits (chave de conteúdo daquele arquivo)
blocos = AES-256-GCM(FK, bloco)
wrapped_FK = AES-256-GCM(KEK, FK) ← guardada no manifesto do arquivo

- **Argon2id** para derivar a chave da senha (resistente a força bruta por GPU).
- **HKDF-SHA256** para separar subchaves por finalidade.
- **AES-256-GCM** para cifrar conteúdo, FKs e metadados (autenticado; nonce aleatório por operação).
- **FK por arquivo:** o conteúdo nunca é cifrado direto com a chave-mestra — só com a FK aleatória, e a FK fica guardada embrulhada pela KEK no manifesto.
- **Deduplicação:** mantida *dentro de um mesmo arquivo* (blocos repetidos não se duplicam). Como cada arquivo tem FK própria, blocos idênticos *entre arquivos diferentes* não deduplicam — custo aceito em troca da rotação barata.

4.2 Rotação da chave-mestra (sem recriptografar arquivos)

Este é o motivo do envelope. Trocar a "chave geral" (nova passphrase e/ou keyfile) gera uma nova `root_key` e, portanto, uma nova `KEK`. A rotação então:

1. desembrulha cada FK com a **KEK antiga**;
2. re-embrulha cada FK com a **KEK nova**;
3. reescreve apenas os manifestos (material de chave, pequeno) e incrementa o *epoch* da KEK.

Os **blocos de conteúdo — o volume pesado — não são tocados**. A rotação é rápida e barata mesmo com muitos GB no cofre. Benefícios adicionais: comprometer uma FK expõe apenas *um* arquivo (raio de dano limitado) e abre caminho para compartilhar/revogar um arquivo isolado no futuro.

Regra inviolável: a passphrase e o keyfile **nunca** são gravados em arquivo de configuração (nem no `.env`). A passphrase é digitada a cada sessão; o keyfile é um arquivo separado, idealmente em mídia removível. Gravá-los anularia a criptografia ponta a ponta.

4.3 Material de chave: sempre trafega, sempre cifrado

Os **manifestos** carregam os metadados (nomes, caminhos), a **FK embrulhada** (`wrapped_FK`) e, **por versão**, o carimbo de data/hora, o hash e as referências aos blocos. Eles são versionados no

servidor e transmitidos normalmente em *toda* sincronização — o que é seguro, porque estão cifrados com a **sua** chave (KEK / manifest_key), que o servidor nunca possui.

Forma lógica do manifesto (em claro no cliente; cifrado no servidor):

```
{
  "path": "Documentos/contrato.odt",
  "wrapped_FK": "<FK embrulhada pela KEK>",
  "versions": [
    {
      "ts":      "2026-06-03 16:42:07",    // data/hora/min/seg – desempate
      "hash":    "blake3:ab12...",        // detecta modificacao
      "size":    184320,
      "blocks":  ["b:9f3...", "b:1c7..."] // referencias aos blocos
    },
    { "ts": "2026-06-01 09:15:33", "hash": "blake3:77ef...", "size": 180224,
      "blocks": ["b:9f3...", "b:0aa..."] }
  ]
}
```

- **Desempate por ts:** em conflito, a versão com carimbo de data/hora mais recente prevalece (pressupõe relógios sincronizados por NTP — todas as máquinas são Debian).
- **Deteção de modificação por hash:** o status de um arquivo vem de comparar o hash atual local com o último hash registrado no histórico de versões do manifesto.
- **Mapeamento bloco ↔ arquivo ↔ versão vive só dentro do manifesto cifrado.** O servidor guarda os blocos como blobs opacos e não sabe a qual arquivo ou versão cada um pertence.
- O servidor guarda e transporta **dois** tipos de dado, ambos cifrados: o conteúdo (blocos cifrados pela FK) e o material de chave + índice (manifestos cifrados pela KEK). Cego para os dois.
- Para abrir um arquivo, o cliente baixa o manifesto, **desembrulha a FK com a KEK local** e só então decifra os blocos.
- A **passphrase e o keyfile** (que geram a KEK) **nunca trafegam** — são a única peça que jamais chega ao servidor.

4.4 Estrutura do manifesto

O manifesto tem dois níveis: um **cabeçalho de cofre** (global) e um **manifesto por arquivo**. Tudo abaixo está em claro apenas no cliente; no servidor é gravado cifrado.

Cabeçalho do cofre (um só):

Campo	Para quê
-------	----------

<code>format_version</code>	Versão do formato do manifesto (evolução futura).
<code>vault_id</code>	Identificador opaco do cofre.
<code>argon2_salt</code>	Salt para derivar a chave da passphrase.
<code>kek_epoch</code>	Geração atual da KEK — base da rotação (4.2).
<code>vault_version</code>	Contador global; sobe a cada alteração. É o valor do <i>compare-and-swap</i> de concorrência no servidor.
índice	Lista dos manifestos de arquivo (<code>file_id</code> → manifesto).

Manifesto por arquivo:

Campo	Para quê
<code>file_id</code>	Id estável e opaco do arquivo.
<code>path</code>	Caminho relativo dentro do cofre.
<code>mode (opcional)</code>	Permissões POSIX, para restaurar idênticas na outra máquina.
<code>wrapped_FK</code>	A chave do arquivo, embrulhada pela KEK.
<code>wrapped_by_epoch</code>	Qual geração da KEK embrulhou a FK (casa com <code>kek_epoch</code> na rotação).
<code>versions[]</code>	Histórico de versões (abaixo).

Cada versão (`versions[]`):

Campo	Para quê
<code>ts</code>	Data/hora/min/seg — critério de desempate .
<code>hash</code>	BLAKE3 do conteúdo — detecção de modificação e integridade.
<code>size</code>	Tamanho em bytes.

blocks Lista **ordenada** de `block_id` que compõem a versão.

device Máquina que criou a versão (rotula conflitos, ajuda no histórico).

deleted Marcador de **tombstone**: apagar vira uma versão "deletada" (some para todos, mas o histórico fica até a limpeza manual).

Exemplo consolidado (forma lógica, em claro no cliente):

```
{
  "file_id": "f:7c1a...",
  "path": "Documentos/contrato.odt",
  "mode": "0644",
  "wrapped_FK": "<FK embrulhada pela KEK>",
  "wrapped_by_epoch": 3,
  "versions": [
    { "ts": "2026-06-03 16:42:07", "hash": "blake3:ab12...", "size": 184320,
      "device": "notebook-casa", "blocks": ["b:9f3...", "b:1c7..."] },
    { "ts": "2026-06-01 09:15:33", "hash": "blake3:77ef...", "size": 180224,
      "device": "pc-trabalho", "blocks": ["b:9f3...", "b:0aa..."] },
    { "ts": "2026-06-04 10:02:11", "deleted": true, "device": "pc-trabalho" }
  ]
}
```

O `block_id` é o HMAC do bloco em claro (deduplicação dentro do arquivo). O *nonce* e a *tag* de cada bloco ficam no próprio blob cifrado no servidor, não no manifesto.

4.5 O que o servidor NÃO vê

- Conteúdo dos arquivos (só blobs cifrados).
- Nomes de arquivos e estrutura de diretórios (vão dentro do manifesto cifrado).
- A passphrase, o keyfile ou a `root_key`.

O servidor enxerga apenas: identificadores opacos de blob, bytes cifrados, o blob do manifesto e um número de versão monotônico usado para detectar concorrência.

Exceção deliberada: o **salt do Argon2** viaja **em claro** no cabeçalho do manifesto (formato *wire*: `{argon2_salt, sealed}`). O salt é parâmetro **público** de KDF (não-secreto, como um salt em `/etc/shadow`) e fica visível para que uma máquina nova derive as chaves e abra o cofre sem cópia manual. A segurança continua nos **dois fatores** (passphrase + keyfile, que nunca vão ao servidor); adulterar o salt em claro só impede a abertura (negação de serviço), não revela conteúdo.

4.6 Acesso remoto por Tor

- O `unuserd` roda como **Onion Service v3** (serviço oculto do Tor).
- O cliente conecta via **proxy SOCKS5 do Tor** (`127.0.0.1:9050`) ao endereço `<...>.onion:porta`.
- O endereço `.onion` é **autoautenticável** (derivado da chave pública do servidor) → protege contra MITM mesmo sem autoridade certificadora. O **mTLS continua** para autenticar o cliente.
- **Resolve o NAT sem IP público nem port-forward** — ideal para máquinas atrás de roteador.
- Custo: exige o pacote `tor` instalado; maior latência — aceitável, pois a sync é manual e transfere só os blocos alterados.
- **VPN/WireGuard: descartado**. Tor é o único mecanismo de acesso fora da LAN.

5. Interface visual (Windows XP Explorer / Luna)

O app reproduz a identidade do Explorer do Windows XP, implementada com **Qt Style Sheets (QSS)** no PySide6. Elementos:

- **Painel de tarefas à esquerda**, com seções recolhíveis de cabeçalho em gradiente azul: *Tarefas de arquivo e pasta*, *Outros locais*, *Detalhes*.
- **Árvore de pastas** + área de itens com ícones grandes.
- Cromagem azul/verde do tema **Luna** (barras e cabeçalhos arredondados).
- As 6 cores de status e as 4 ações aparecem no painel lateral e no menu de contexto, no estilo XP.

A reprodução do visual é estética/homenagem. **Não** serão usados ícones ou imagens proprietários da Microsoft — equivalentes próprios serão recriados.

6. Componentes

Binário	Onde roda	Papel
<code>unuser</code> (GUI)	Cada máquina cliente	App de cofre estilo XP Explorer. Destrava com passphrase+keyfile, compara local × servidor, mostra status e executa as ações.
<code>unuserd</code>	Nó central	Cofre cego. Armazena blobs + manifesto cifrados, mantém histórico (sempre ligado) de versões, autentica clientes por mTLS, publica Onion Service para

acesso via Tor.

7. Fluxo do cliente

1. **Destruar:** abrir o app, informar passphrase + keyfile → deriva a root_key.
2. **Carregar config:** lê o `.env` (conexão/flags) e o JSON com os diretórios padrão (recursivos) + itens avulsos marcados.
3. **Conectar:** usa o modo escolhido na interface (IP direto ou Tor).
4. **Comparar:** faz o scan local (hash dos arquivos, respeitando o `.unuserignore`) e busca o manifesto cifrado do servidor.
5. **Exibir:** mostra a árvore tipo Explorer, cada arquivo com seu status.
6. **Decidir:** por arquivo (ou seleção múltipla) o usuário escolhe a ação.
7. **Executar:** Send/Receive aplicam as ações marcadas.

7.1 Status por arquivo

Status	Significado
Em sincronia	Local e servidor idênticos.
Modificado no local	O arquivo local difere da versão do servidor.
Modificado no servidor	O servidor tem uma versão mais nova que a local.
Conflito	Mudou nos dois lados desde a última sincronização.
Só local	Existe na máquina, nunca foi enviado.
Só no servidor	Existe no cofre, ainda não foi baixado.

7.2 Ações por arquivo

#	Ação	Efeito
1	Substituir no servidor	Envia a versão local, sobrescrevendo a do servidor.
2	Substituir no local	Baixa a versão do servidor, sobrescrevendo a local.
3	Versionar	Guarda a versão anterior no histórico em vez de sobrescrever.

4 Ignorar

Não faz nada neste ciclo.

8. Regras de negócio

- **Diretórios são recursivos:** todo o conteúdo abaixo de uma raiz do JSON entra no cofre.
- **Renomear** = criar arquivo novo + apagar o antigo (não há tratamento especial de rename).
- **Apagar** apaga para todos, inclusive no servidor — mas a versão anterior fica preservada no histórico, evitando perda acidental irreversível.
- **Versões** são guardadas indefinidamente; a limpeza é **manual** (o usuário decide quando apagar histórico). Alertas de acúmulo ficam para uma fase futura.

9. Configuração e arquivos locais

Princípio: um arquivo por responsabilidade, cada um no formato que combina com ela — sem duplicação e sem nenhum arquivo fazendo dois papéis. Toda a edição acontece pela GUI; os arquivos são apenas o reflexo persistido. **Nenhum deles contém segredos** (passphrase/keyfile ficam de fora — ver 4.1).

Arquivo	Responsabilidade	Formato (porquê)	Se faltar
<code>~/.env</code>	Como alcançar o servidor: conexão e parâmetros escalares de runtime (modo IP/Tor, host, porta, onion, socks).	CHAVE=VALOR — são valores escalares simples, que é exatamente o que o <code>.env</code> expressa bem.	App pede a conexão na GUI no primeiro uso e grava o arquivo.
<code>~/.config/unuser/dirs.json</code>	O que vai no cofre: diretórios padrão (recursivos) + itens avulsos.	JSON — é uma <i>lista estruturada</i> com objetos/flags, que não cabe bem em CHAVE=VALOR.	Seleção vazia; o usuário adiciona pastas pela GUI.

<code>~/ .unuserignore</code>	O que NÃO vai no cofre: exclusões da varredura recursiva.	Globs estilo <code>.gitignore</code> Criado com — formato consagrado defaults de para padrões de segurança no exclusão. primeiro uso.
-------------------------------	---	--

A separação não é arbitrária: *conexão*, *seleção de conteúdo* e *exclusões* são três preocupações distintas, e cada formato (escalar / lista estruturada / globs) é o natural para a sua. Não há valor repetido entre os arquivos.

9.1 `~/ .env` — conexão

```
UNUSER_VAULT_ID=<identificador-opaco-do-cofre>
UNUSER_CONN_MODE=direct          # direct | tor
UNUSER_DIRECT_HOST=192.168.0.10
UNUSER_DIRECT_PORT=8443
UNUSER_TOR_ONION=<...>.onion
UNUSER_TOR_PORT=8443
UNUSER_TOR_SOCKS=127.0.0.1:9050
```

9.2 `~/ .config/unuser/dirs.json` — seleção de conteúdo

```
{
  "default_dirs": [
    { "path": "/home/usuario/Documentos", "recursive": true },
    { "path": "/home/usuario/Projetos",   "recursive": true }
  ],
  "extra_items": [
    "/home/usuario/notas/ideias.md"
  ]
}
```

9.3 `~/ .unuserignore` — exclusões (sempre presente)

Existe **sempre**, pois a sync é recursiva e sem ele é fácil varrer segredos sem querer. Nome com namespace para não conflitar com o `.ignore` de outras ferramentas (ripgrep, fd). Defaults de segurança embutidos:

```
# defaults de segurança (embutidos)
.env
*.key
*.pem
.ssh/
```

```
.gnupg/  
*.tmp  
*.swp  
.git/  
# o usuario pode adicionar mais pela interface
```

10. Servidor cofre cego

- **Armazenamento:** blobs imutáveis endereçados por id opaco + manifesto cifrado versionado.
- **API (sobre mTLS, direta na LAN ou via Tor):** obter/enviar manifesto (com *compare-and-swap* na versão), obter/enviar blob, listar histórico.
- **Concorrência:** se dois clientes tentam atualizar o manifesto, o segundo recebe rejeição por versão desatualizada → o cliente re-busca e o conflito aparece como status na interface.
- **Coleta de lixo:** blobs não referenciados por nenhum manifesto/versão podem ser removidos na limpeza manual.

11. Roadmap em fases

Fase	Entrega
------	---------

- | | |
|---|--|
| 1 | Núcleo de criptografia (derivação de chaves, cifra/decifra, block-id) + testes. |
| 2 | Chunker + índice local (SQLite, hash de blocos). |
| 3 | Manifesto (serialização, cifragem, versionamento). |
| 4 | Servidor cofre cego (API de blobs/manifesto, mTLS, Onion Service). |
| 5 | App cliente PySide6 com visual XP (árvore, status, 4 ações, send/receive, escolha IP/Tor). |
| 6 | Empacotamento (.deb/systemd, instalação nas máquinas). |

12. Em aberto / futuro

- Modo de conexão "auto" (tenta IP na LAN e cai para Tor) — opcional, hoje a escolha é

manual.

- Alertas de acúmulo de versões.
- Sincronização do próprio arquivo de configuração entre máquinas.
- Descoberta local na LAN para transferência direta entre pares.